

LiDAR Scout Rover

Abstract - This paper presents the software architecture, development process, and experimental results of the LiDAR Scout Rover project. The rover was designed as a distributed ROS 2 Jazzy system that combines an Arduino Mega, a Raspberry Pi 5, and a workstation to support motor control, gas sensing, LiDAR publishing, camera display, odometry, mapping, and navigation. At the embedded layer, the Arduino Mega manages low-level drive control and raw gas acquisition. At the edge layer, the Raspberry Pi 5 bridges serial hardware data into ROS 2 topics, publishes the Unitree 4D LiDAR point cloud and IMU streams, and supports camera and network services. At the workstation layer, Point-LIO, Nav2, custom obstacle-processing nodes, RViz2, and browser-based interfaces provide higher-level autonomy and operator awareness. Two navigation pipelines were developed and compared: an initial front-focused version and an improved 360-degree version for broader obstacle coverage. Experimental results show that the final system can drive, publish gas data, display camera feedback, build LiDAR-based maps, and perform semi-autonomous navigation while maintaining a modular software structure that can be extended in future work.

I. Introduction

The LiDAR Scout Rover project was developed to investigate how a compact mobile robot can combine embedded control, real-time sensing, and high-level autonomy into a single practical platform. The original proposal emphasized several software goals: driving the rover in all directions, reading gas sensor data, displaying camera data, publishing LiDAR data, mapping the surrounding environment, and avoiding obstacles. Meeting all of these objectives on one rover required more than a single program. It required a complete software pipeline that could move data and commands across multiple computers while maintaining timing, topic consistency, and operator visibility.

The final rover architecture therefore evolved into a distributed ROS 2 Jazzy system. Low-level functions were assigned to an Arduino Mega and a Raspberry Pi 5, while higher compute functions such as Point-LIO odometry, Nav2 path planning, RViz2 visualization, and browser-based telemetry were handled on a workstation. This separation was necessary because the LiDAR, odometry, and navigation components demanded more processing power than was comfortable to keep on the rover alone,

especially during live demonstrations. The result was a layered robotics platform in which each computer handled a specific class of tasks while ROS 2 topics connected the full system.

Although the main focus of this paper is the rovers software architecture, the system also required mechanical and power integration to support reliable testing. The LiDAR, Raspberry Pi 5, camera, gas sensors, batteries, converters, and wiring had to be mounted on a compact tracked chassis in a way that kept the rover stable, accessible, and serviceable. Custom 3D-printed parts were used to improve the physical layout and support repeated hardware changes during development.

II. System Architecture

The software architecture of the LiDAR Scout Rover was organized into four connected layers: an embedded layer, an edge bridge layer, a workstation autonomy layer, and a browser visualization layer. This arrangement allowed the system to stay modular while still supporting a complete sensing and navigation workflow. At the embedded layer, an Arduino Mega 2560 was used to control the two DC motors and sample gas sensor data. The Arduino was connected to a motor driver shield and to two gas sensors, the MQ-2 and MQ-3. Its software responsibilities were intentionally kept simple. It received serial motion commands from the Pi 5 in the form "v <linear> <angular>", converted those commands into motor outputs, sampled raw analog gas values, and returned the values over serial as "g <mq2_raw> <mq3_raw>". Keeping the Arduino simple reduced latency in motor control and limited the amount of logic running at the lowest hardware level.

At the edge bridge layer, the Raspberry Pi 5 served as the hardware-to-ROS interface. A Python bridge node subscribed to /cmd_vel, converted the Twist command into the serial motor protocol expected by the Arduino, and also published raw gas readings after parsing the returned serial frames. The Pi 5 additionally hosted the Unitree 4D LiDAR L1 driver, which published the point cloud and IMU topics used by the rest of the system. A camera publisher and browser support services were also assigned to the Pi side. In practice, this meant the Pi 5 became the main sensor and device gateway for the rover.

At the workstation layer, a laptop or Intel NUC ran the higher-level robotics stack. This computer hosted Point-LIO for LiDAR-inertial odometry, Nav2 for path planning and

controller execution, RViz2 for debugging and visualization, custom nodes for obstacle processing and gas-data conversion, and optional rosbridge and web streaming services for the dashboard. This split proved especially useful because Point-LIO, costmaps, and RViz2 were significantly heavier than the low-level bridging tasks on the Pi 5.

At the browser layer, a website was used for operator awareness and remote status monitoring. The site displayed navigation state, selected topic status, and camera output, and it was designed to work through rosbridge and local HTTP services. Although the website still has room for refinement, it helped make the system easier to operate than relying entirely on terminal output alone.

The complete data and command flow can be summarized as follows. A motion command generated by Nav2, the web interface, or teleoperation flows through a command mux and a collision monitor to the final `/cmd_vel` topic. The Pi bridge reads that topic and forwards the corresponding serial motor command to the Arduino. In the opposite direction, the gas sensors are sampled by the Arduino, relayed by the Pi into ROS 2 raw gas topics, and then converted into higher-level gas status topics on the workstation. Simultaneously, the LiDAR and IMU data stream from the Pi to the workstation, where Point-LIO and Nav2 consume them for odometry, obstacle processing, and planning.

The physical architecture also supported the software architecture by keeping major components organized on the tracked chassis. Custom parts designed in Fusion 360, including a front plate and skid plate, were 3D printed using PLA filament. These parts helped provide cleaner mounting surfaces for exterior hardware and improved the rover's overall serviceability during testing. The mounting layout was especially important because the rover used multiple power rails, external sensing devices, and networked computing hardware that needed to remain secure while the platform was moving.

III. Software Development and Integration

The development process followed a staged software integration strategy. Rather than attempting to build the full rover stack all at once, each subsystem was verified independently before it was merged into the larger ROS 2 pipeline. This approach reduced debugging complexity and made it easier to isolate timing and interface errors.

The first software stage focused on embedded control. The Arduino Mega was programmed to handle motor control and gas acquisition. Its motor code was designed to remain close to the original known-good behavior so the rover would not lose responsiveness when additional gas functionality was introduced. The final sketch therefore emphasized sending data rather than performing advanced interpretation. Raw MQ-2 and MQ-3 values were transmitted upward, leaving voltage conversion and threshold logic to higher software layers.

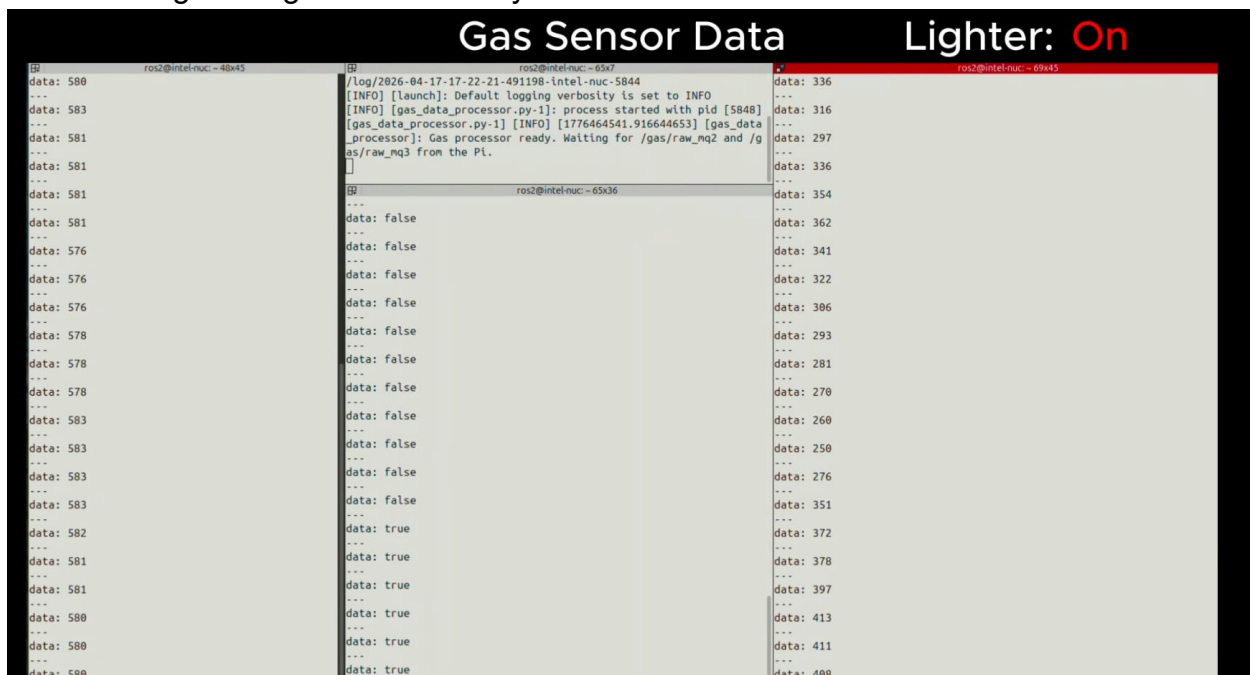


Figure 1 – Gas Sensors Data

The second stage focused on the Raspberry Pi 5 bridge. The Pi node was responsible for owning the serial link to the Arduino, subscribing to /cmd_vel, forwarding motion commands, and publishing raw gas topics. This was an important design choice because it prevented two separate nodes from fighting over the same serial port and kept motor traffic and sensor traffic within one coordinated process. In addition to the Arduino bridge, the Pi 5 hosted the Unitree LiDAR driver and camera publisher so that the rover itself always originated the raw sensing topics.

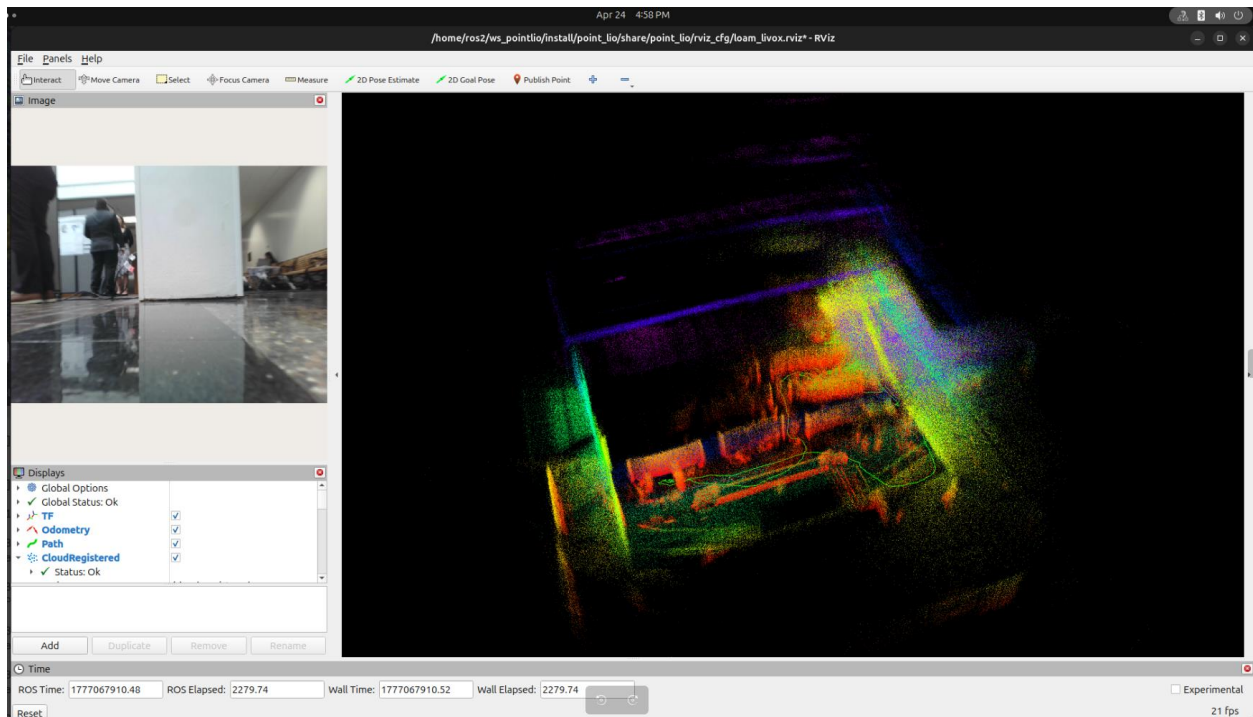


Figure 2 – Camera + /cmd_vel (xbox controller) + Point-Lio

The third stage focused on odometry and mapping. Point-LIO was selected because it provided a strong LiDAR-inertial odometry solution for the Unitree LiDAR stream. After configuration and topic alignment, the workstation could use the Pi-published point cloud and IMU streams to estimate odometry and support mapping. This gave the rover a working spatial reference that could later feed the navigation stack.

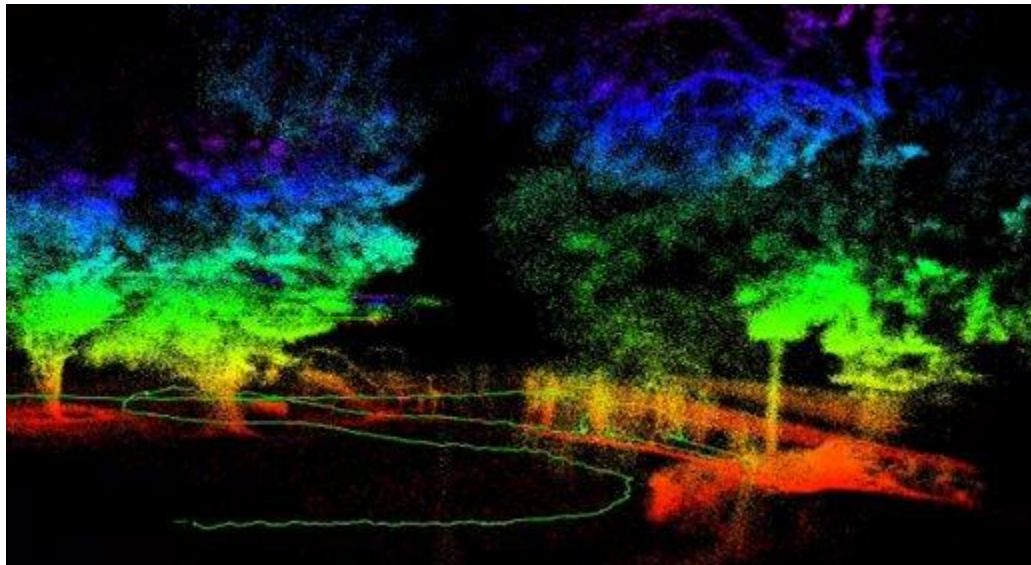


Figure 3 – Point-Lio Outside of Delta

The fourth stage focused on navigation. Two workspaces were used during development: ws_lsr_nav2 and ws_lsr_nav2_v2. The first workspace was the original navigation pipeline and emphasized a more forward-biased obstacle view. That version was simpler and easier to reason about early in development, but it was limited when the rover reversed or encountered obstacles from the side or rear. The second workspace, ws_lsr_nav2_v2, introduced a 360-degree obstacle cloud for Nav2 costmaps, a larger robot footprint, additional safety handling, and a cleaner separation between full-environment costmap processing and short-range reflex avoidance. V2 therefore became the stronger long-term navigation path.

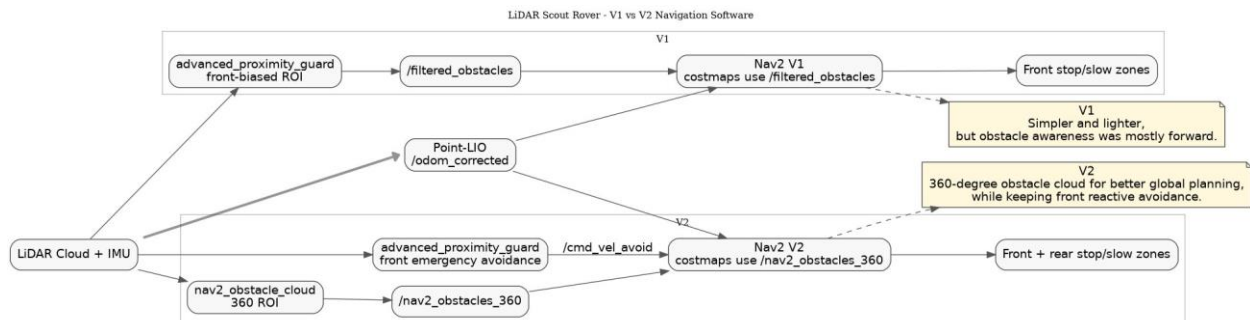


Figure 4 – Point-LIO & NAV 2 System Flowchart

The fifth stage focused on the web interface and operator tools. Browser connectivity was added through rosbridge and a local web server so that status, camera data, and topic-level awareness could be displayed without depending exclusively on terminals. This was particularly useful during demonstrations, because it provided a simplified operator view of the same system that RViz2 and ROS 2 nodes were using underneath.

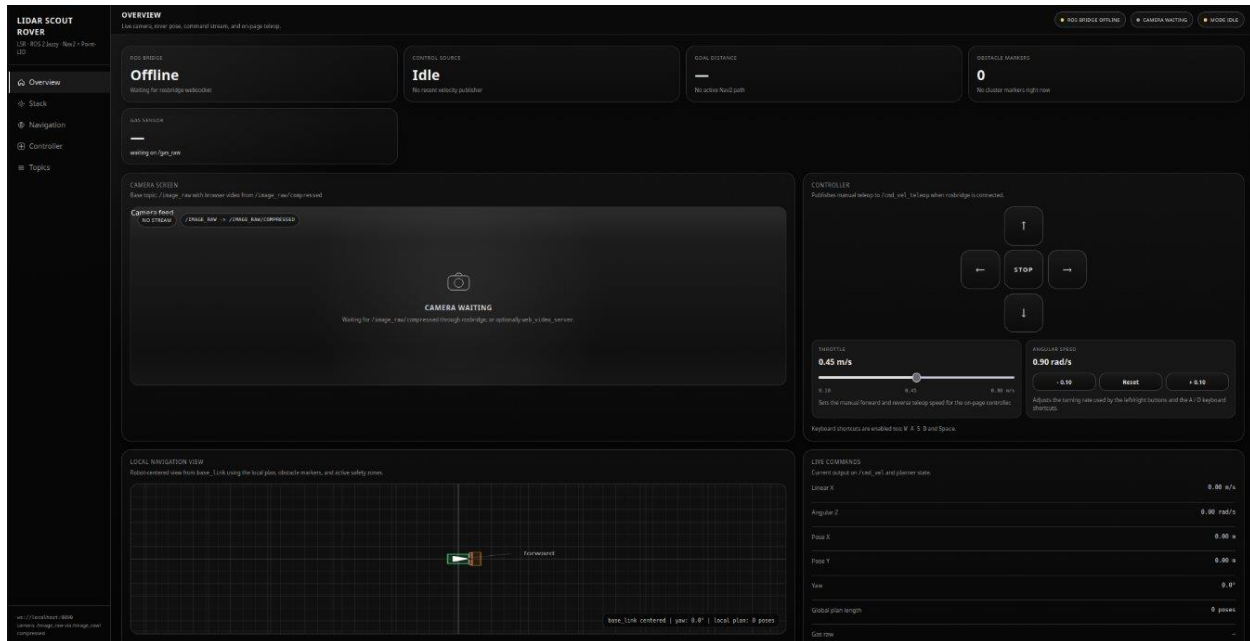


Figure 5 – Website UI

Several mathematical and logical transformations occur throughout this software pipeline. For gas sensing, the Arduino reports raw analog-to-digital converter values. These are later converted into voltages on the workstation using a standard linear conversion of the form $V = (\text{raw} / 1023) * V_{\text{ref}}$, where V_{ref} is the ADC reference voltage. Alert topics are then generated by comparing the processed values to configured thresholds. For motor control, linear and angular commands are mapped to left and right track motion by the Arduino-side control logic, allowing a single Twist command to produce forward, reverse, or turning behavior. For navigation, the costmap pipeline converts the LiDAR point cloud into filtered obstacle representations through cropping, voxelization, and ground removal so that only relevant obstacles are retained for planning.

IV. Testing and Analysis

Testing and debugging were carried out continuously during development. Because the rover used multiple computers and several custom ROS 2 nodes, functional success depended not only on whether each node ran, but also on whether the timing and topic interactions remained stable under load.

Initial tests verified the communication path between the Arduino and the Pi. The motor bridge was tested by observing whether `/cmd_vel` commands produced the expected serial messages and whether the rover responded correctly in forward, reverse, and turning modes. The gas pipeline was tested in stages by first confirming raw gas lines on the serial interface, then confirming raw ROS 2 gas topics from the Pi, and finally confirming processed gas topics on the workstation. This staged method made it possible to verify each conversion step separately.

The LiDAR path was tested by validating the point cloud and IMU topics from the Pi, then confirming that Point-LIO consumed them correctly and produced stable odometry. Once that worked, Nav2 and RViz2 were brought into the loop. At this stage, the main challenge was not whether the software launched, but whether it remained stable under the combined load of point cloud rendering, odometry, obstacle processing, navigation, and browser services.

This challenge became especially clear during live demonstrations. When too many heavy processes were active at once, the rover began to experience queue overflows, stale obstacle warnings, RViz2 instability, and odometry cutoffs that could trigger navigation resets. Several software-level changes were therefore introduced to improve stability. The NUC was switched from balanced mode to performance mode. Visualization load was reduced or moved off the robot computer when possible. Topic monitoring was used to identify stale streams and timing mismatches. The navigation stack was also split into V1 and V2 versions so different costmap strategies could be compared directly.

The comparison between V1 and V2 was one of the most informative parts of testing. V1 used a more front-focused obstacle-processing strategy, which sometimes felt simpler and visually steadier. However, it was limited whenever the rover backed up or when significant obstacles appeared outside the forward field of view. V2 added a separate 360-degree obstacle cloud for Nav2 costmaps while still preserving a custom short-range front safety layer. This improved the planner's awareness of the surrounding environment and made the rover more capable in complex indoor spaces, although it also made timing and computation more demanding.

Testing also included user-facing operation. The system was evaluated on how quickly it could be brought online, whether the Pi correctly auto-booted its bridge responsibilities, whether the workstation could discover the rover topics on the shared ROS domain, and whether the website and RViz2 could both reflect live rover state.

This form of usability testing was important because a technically correct system that takes too long to start or requires too many manual corrections is difficult to demonstrate reliably.

V. Results and Future Work

The final software stack achieved the primary functional goals of the LiDAR Scout Rover project. The rover could drive in commanded directions, publish gas data, display camera output, generate LiDAR-based maps with Point-LIO, and perform obstacle-aware navigation using Nav2. The system also demonstrated that a distributed ROS 2 design can make practical use of an Arduino Mega, a Raspberry Pi 5, and a workstation without losing the ability to coordinate motion and sensing in real time.

From a navigation perspective, the development of two workspaces led to a clear result. The original V1 pipeline provided an important baseline and was useful for early experiments, but the V2 pipeline was superior for broader navigation because it provided 360-degree obstacle awareness for costmap generation. This improvement was especially important when the rover was not strictly moving forward, since side and rear obstacle awareness became much more relevant in realistic environments.

From a system-integration perspective, the results also showed the importance of separating responsibilities across hardware layers. The Arduino remained responsive because it was not overloaded with ROS logic. The Pi 5 acted as a clean bridge between hardware and ROS 2 topics. The workstation carried the heavier odometry and planning workload. This separation helped the rover evolve from a collection of sensors and scripts into a more organized robotic platform.

Several improvements remain for future work. First, TF drift and odometry stability should be further reduced so the rover can finish navigation goals with greater precision. Second, the V2 stack should continue to be tuned for approach speed, footprint behavior, and obstacle timing so the rover performs more consistently near goals. Third, gas thresholds should be calibrated more carefully so the published alert states better reflect the actual environmental conditions. Fourth, the website should be updated to fully match the latest gas and navigation topic layout. Finally, more automation should be added to the startup process so demonstrations require fewer manual steps and fewer network-specific adjustments.

Future mechanical improvements could include stronger printed materials, cleaner cable routing, and a more modular mounting system so sensors, batteries, and electronics can be replaced or repositioned more quickly during testing.

Overall, the project succeeded in building a software architecture that supports sensing, visualization, and autonomous rover behavior in one integrated ROS 2 system. That result provides a strong base for continued refinement toward a more reliable field-ready scout rover.

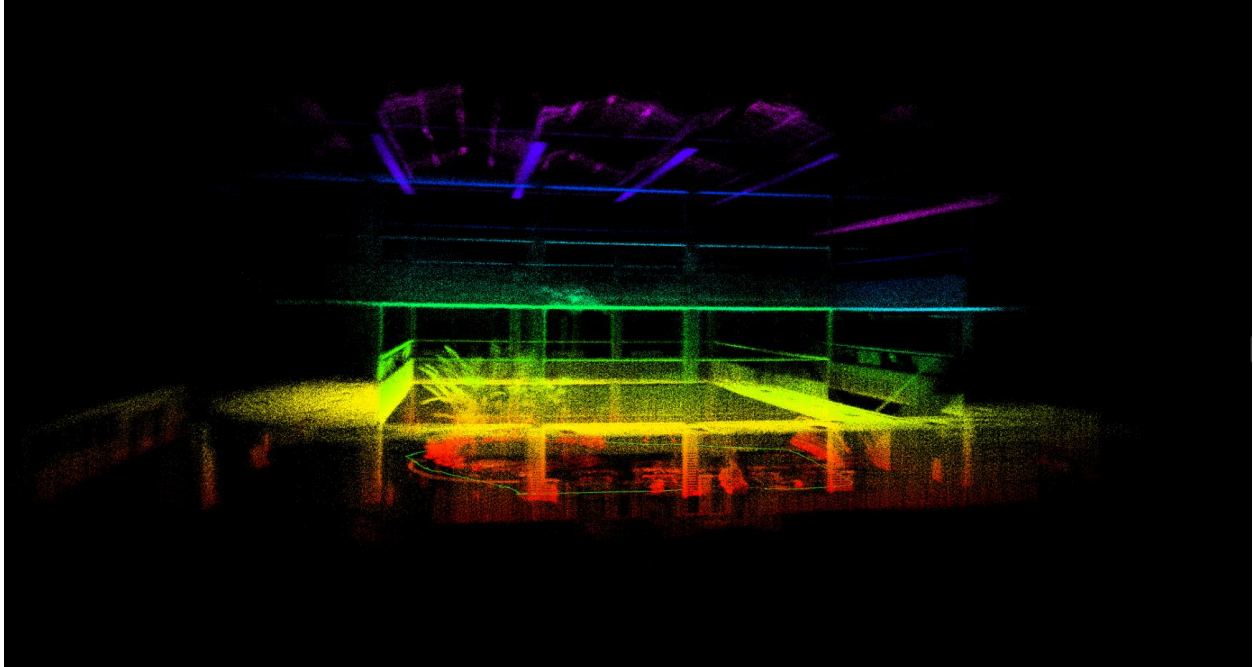


Figure 6 – Point-Lio UHCL Bayou Building

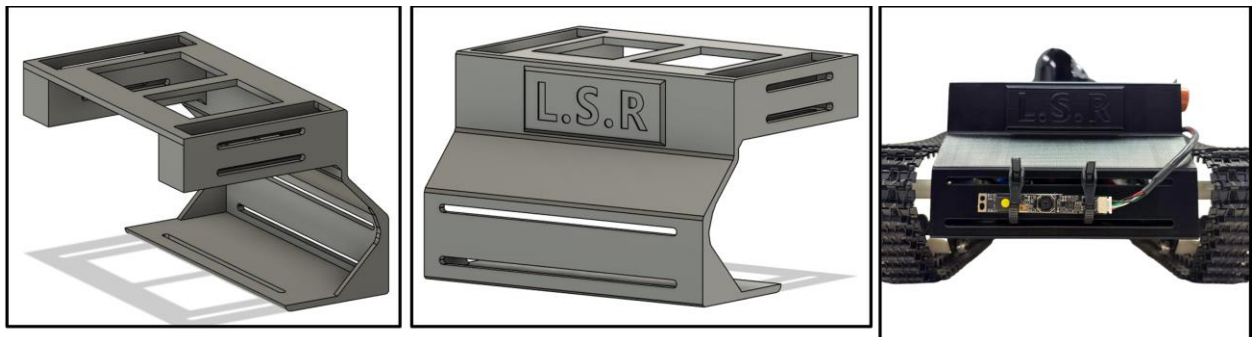


Figure 7 – 3D-printed Front Plate

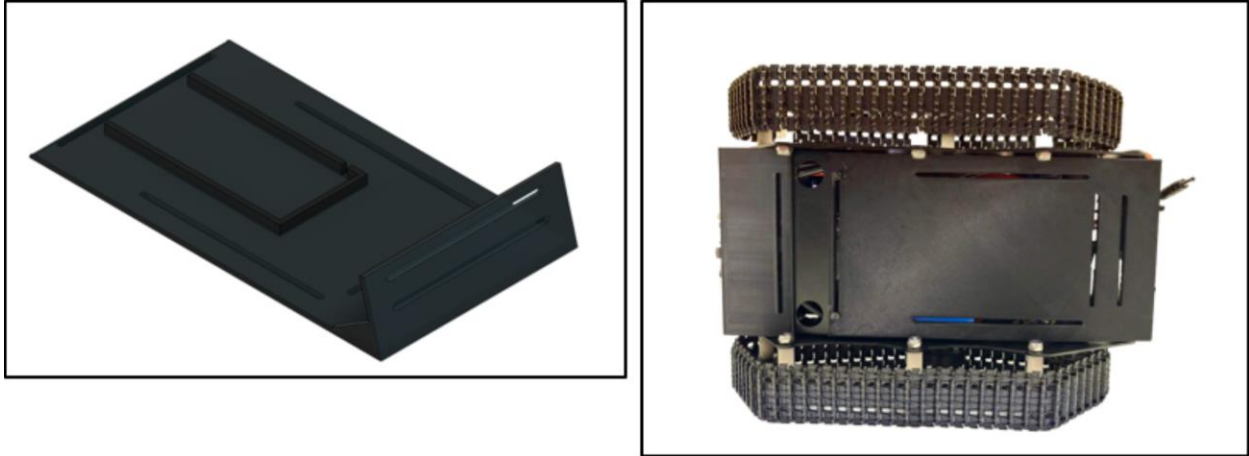


Figure 8 – 3D-printed Skid Plate

VI. References

- [1] Amazon.com: Swaytail Premium Large Metal 4WD shock absorption robot tank car chassis kit for Arduino/Raspberry Pie/MicroBit with 33 DC motor, shock absorbing tracked crawler with damping system DIY Maker Model : Toys & Games, <https://www.amazon.com/Premium-Absorption-Raspberry-MicroBit-Absorbing/dp/B09CYDB8H4> (accessed Nov. 19, 2025).
- [2] Amazon.com: For Raspberry Pi 5 8GB starter kit includes with Raspberry Pi 5 8GB, PI5 case with active cooler, 27W USB type-C power supply, 64GB SD card and TF Card Reader, HDMI cable, ethernet cable (8 items) : Electronics, <https://www.amazon.com/Raspberry-Starter-Active-Cooler-Ethernet/dp/B0CXXFFYNB> (accessed Nov. 19, 2025).
- [3] Walmart.com, L293D Motor Drive Shield Expansion Board For Arduino Uno Mega2560 & Duemilanove, Envistia Mall, <https://www.walmart.com/ip/L293D-Motor-Drive-Shield-Expansion-Board-For-Arduino-Uno-MEGA2560-Duemilanove/400667780?wmlspartner=wlpa&selectedSellerId=101028229> (accessed Apr. 12, 2025).

[4] "4D Lidar L1 application Scenarios_4D LIDAR L1 efficacy: Unitree Robotics," 4D Lidar L1 Application Scenarios_4D Lidar L1 Efficacy | Unitree Robotics, <https://www.unitree.com/LiDAR/> (accessed Nov. 19, 2025).

[5] Amazon.com: Arducam IMX219 8MP Autofocus USB Camera Module, 3280(H)x2464(V)@30fps with Single Microphone for Linux, Windows, and Mac OS, <https://a.co/d/0fThY8lZ> (accessed Apr. 4, 2026).

[6] Amazon.com: CJRSLRB® Gas Detection Module MQ-2 MQ-3 MQ-4 MQ-5 MQ-6 MQ-7 MQ-8 MQ-9 MQ-135 sensor module each of them 1pcs total 9pcs sensor kit : Tools & Home Improvement, <https://www.amazon.com/CJRSLRB-Detection-Module-MQ-135-Sensor/dp/B016KABTDK> (accessed Nov. 19, 2025).

[7] Mavdrones, "Tattu 2200 mah lipo battery with XT90-S Anti-Spark connector," Mavdrones, <https://www.mavdrones.com/product/tattu-22-2v-25c-6s-2200-mah-lipo-battery-with-xt90-s-anti-spark-connector/> (accessed Nov. 19, 2025).

[8] Amazon.com: BrosTrend 650mbps linux compatible WIFI adapter supports Kali Linux, ubuntu, Mint, debian, kubuntu, zorin, pureos, Raspberry Pi 2+, windows, dual band USB Wireless Adapter W/ long range WIFI antenna : Electronics, <https://www.amazon.com/BrosTrend-600Mbps-Wireless-Internet-AC2/dp/B01GC8XH0S> (accessed Nov. 19, 2025).

[9] E. Ackerman, "Boston Dynamics' Spot Robot Dog now available for \$74,500," IEEE Spectrum, <https://spectrum.ieee.org/boston-dynamics-spot-robot-dog-now-available> (accessed Nov. 19, 2025).

[10] Base Year:2024|, "Autonomous Mobile Robots Market Size (\$13.7 billion) 2030," STRATEGIC MARKET RESEARCH, <https://www.strategicmarketresearch.com/market-report/autonomous-mobile-robots-market> (accessed Nov. 19, 2025).

[11] D. He, W. Xu, N. Chen, F. Kong, C. Yuan, and F. Zhang, "Point-LIO: Robust High-Bandwidth Light Detection and Ranging Inertial Odometry," *Advanced intelligent systems*, vol. 5, no. 7, Apr. 2023, doi: <https://doi.org/10.1002/aisy.202200459>.

[12] Adam Zewe | MIT News, "Engineers enable a drone to determine its position in the dark and indoors," MIT News | Massachusetts Institute of Technology, <https://news.mit.edu/2025/engineers-enable-drone-determine-its-position-dark-and-indoors-0213> (accessed Nov. 19, 2025).

[13] "What is a Lithium Polymer Battery? | LiPo," *Ossila*, 2021.

<https://www.ossila.com/pages/lithium-polymer-battery>

[14] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022, doi: <https://doi.org/10.1126/scirobotics.abm6074>.

[15] "ROS 2 Documentation: Jazzy documentation," ROS 2 Documentation, <https://docs.ros.org/en/jazzy/index.html> (accessed Sept. 2025).

[16] "Nav2," Navigation2 Documentation, <https://docs.nav2.org/> (accessed Jan. 2026).

[17] "Costmap 2D," Navigation2 Documentation, <https://docs.nav2.org/configuration/packages/configuring-costmaps.html> (accessed Jan. 2026).

[18] "RViz," ROS 2 Documentation: Jazzy documentation, <https://docs.ros.org/en/jazzy/Tutorials/Intermediate/RViz/RViz-Main.html> (accessed Sept. 2025).

[19] Point Cloud Library (PCL), PointClouds.org, <https://pointclouds.org/> (accessed Apr. 1, 2026).

[20] "ROS 2 Jazzy Jalisco Released!," Open Robotics Discourse, <https://discourse.openrobotics.org/t/ros-2-jazzy-jalisco-released/37862> (accessed Sept. 2025).